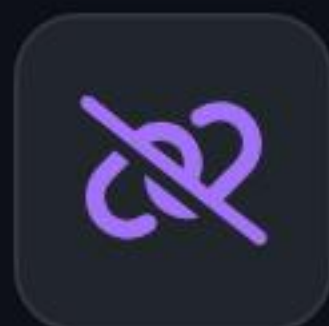


Inyección de Dependencias y Servicios

Arquitectura Limpia en ASP.NET Core y C# 14

Hoja de Ruta



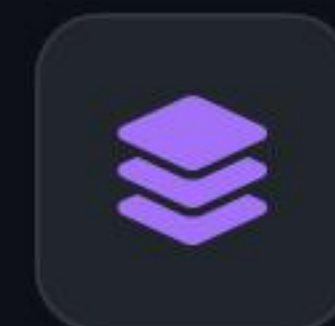
1. El Problema

El acoplamiento fuerte con `new` y el Principio de Inversión de Control (IoC).



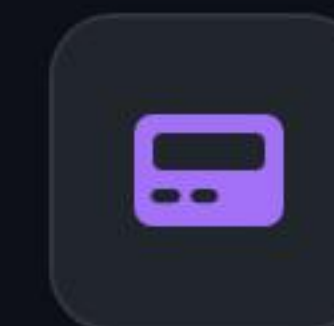
2. El Contenedor DI

Ciclos de vida en .NET: Transient, Scoped y Singleton.
Configuración en `Program.cs`.



3. Servicios y EF Core

Creación de la capa de Servicios, abstracción e integración óptima con bases de datos.



4. Razor Pages

Consumo de servicios desde la UI usando Constructores Primarios (C# 14).

Sección 1:

El Acoplamiento

¿Por qué la palabra reservada 'new' es un problema a gran escala?

El Anti-patrón: Acoplamiento Fuerte

Creación Manual de Instancias

Cuando una clase crea sus propias dependencias utilizando `new`, se "acopla" rígidamente a esa implementación específica. No podemos cambiarla, probarla aisladamente, ni gestionar su ciclo de vida de forma global.

"New is Glue" (New es pegamento). Pega tus clases de tal forma que separarlas luego es un infierno.

```
public class FacturacionPageModel : PageModel
{
    // ✗ MAL: La clase crea sus dependencias
    private readonly EmailSender _emailSender;
    private readonly AppDbContext _db;

    public FacturacionPageModel()
    {
        // Acoplamiento duro. Imposible de testear
        // sin una BD real o un servidor SMTP.
        _emailSender = new EmailSender("smtp.gmail.com");
        _db = new AppDbContext("Data Source=app.db");
    }
}
```


Principio de **Inversión de Control (IoC)**

El Principio de Hollywood

"No nos llames, nosotros te llamaremos". En lugar de que la clase busque o cree sus dependencias, un ente superior (el framework) se las "inyecta" ya construidas.

- ✓ Las clases solo piden lo que necesitan (a través de su constructor).
- ✓ Ignoran cómo se construyen esas dependencias.
- ✓ Permite sustituir implementaciones (ej. usar un MockEmailSender en pruebas).



Inyección de Dependencias (DI)

La DI es la técnica principal para lograr la Inversión de Control. ASP.NET Core trae un contenedor de DI nativo y muy potente integrado desde el minuto cero.

Sección 2:

El Contenedor de ASP.NET Core

Configurando quién fabrica qué objeto

Los 3 Ciclos de Vida (Lifetimes)



Transient

Corta duración: Se crea una instancia nueva *cada vez* que se solicita al contenedor. Útil para servicios ligeros y sin estado (stateless).



Scoped

Por petición: Se crea una única instancia por cada petición HTTP. Todos los componentes en la misma request comparten el mismo objeto. *¡El estándar para EF Core!*



Singleton

Global: Se crea una sola instancia la primera vez y se reutiliza para siempre en toda la aplicación. Ideal para cachés o configuraciones.

Configuración en Program.cs

El Catálogo de Servicios

En el archivo de arranque, configuramos el `IServiceCollection`. Es como enseñarle al framework un diccionario: "Cuando alguien pida X, dale una instancia de Y".

- 📄 Se definen antes del `builder.Build()`.
- 🛡️ Se asocian Interfaces con Clases Concretas.

```
var builder = WebApplication.CreateBuilder(args);

// Registramos servicios indicando su ciclo de vida

// Transient: Instancia nueva cada vez
builder.Services.AddTransient<ITaxCalculator, TaxCalculator>()

// Scoped: 1 por petición HTTP
builder.Services.AddScoped<IOrderService, OrderService>();

// Singleton: 1 para toda la app
builder.Services.AddSingleton<IEmailConfig, GlobalEmailConfig>()

var app = builder.Build();
```


Sección 3:

El Concepto de Servicio

Separando la lógica de negocio de la interfaz de usuario

Separación de Responsabilidades (SRP)

El Problema de los Controladores Gordos

Si pones reglas de negocio, cálculos de impuestos y llamadas directas a la base de datos dentro del `PageModel`, ese código no se puede reutilizar en otra parte (ej: una API o una tarea de fondo).

La Solución: Capa de Servicios

Un **Servicio** es una clase dedicada a contener la lógica pura del negocio. El `PageModel` simplemente captura los datos HTTP y se los pasa al servicio para que haga el trabajo real.

Flujo Moderno: Razor Page → Servicio → EF Core (BD)

1. Definir el Contrato (Interfaz)

Interfaces y DTOs

Para no acoplarnos, el servicio siempre se define mediante una interfaz (IProductService). Además, usa records (DTOs) para la entrada y salida de datos, sin filtrar las entidades directas de base de datos a la UI.

```
// DTO de Salida (Record inmutable de C#)
public record ProductDto(Guid Id, string Name, decimal Price);

// Contrato del Servicio
public interface IProductService
{
    Task<IReadOnlyList<ProductDto>> GetActiveProductsAsync(
        CancellationToken ct = default);

    Task<bool> CreateProductAsync(
        string name, decimal price, CancellationToken ct);
}
```


Sección 4:

Integración con EF Core

Conectando el ORM con nuestra capa de servicios

Registrando EF Core: DbContext Pooling

Rendimiento Extremo

Tradicionalmente usábamos `AddDbContext`. En arquitecturas modernas (C# 14 / .NET 10), usamos **`AddDbContextPool`**. En lugar de crear un `DbContext` nuevo por cada petición, el framework recicla instancias vacías, ahorrando memoria y tiempo de CPU.

Por defecto, se registra con ciclo de vida **Scoped**.

```
var builder = WebApplication.CreateBuilder(args);

//  Registro óptimo para alta concurrencia
builder.Services.AddDbContextPool<AppDbContext>(opt =>
    opt.UseSqlite(
        builder.Configuration.GetConnectionString("Db")
    )
);

// Registramos nuestro servicio asociado a la Interfaz
builder.Services.AddScoped<IProductService, ProductService>();
```


Implementando el Servicio (C# 14)

```
//  Constructor Primario: DI directa sin campos privados man
public class ProductService(AppDbContext db, ILogger<ProductSe
    : IProductService
{
    public async Task<IReadOnlyList<ProductDto>> GetActiveProd
        CancellationToken ct = default)
    {
        logger.LogInformation("Consultando productos activos..

        return await db.Products
            .AsNoTracking() // Clave para consultas de lectura
            .Where(p => p.IsActive)
            .Select(p => new ProductDto(p.Id, p.Name, p.Price)
            .ToListAsync(ct);
    }
}
```

Constructor Primario

En C# 14, inyectamos dependencias (el `AppDbContext` y el `ILogger`) directamente en la declaración de la clase. El framework las provee automáticamente al crear el servicio.

Nota: Nunca cargues la entidad entera para leer; proyecta directamente al `ProductDto` con `Select()` y usa `AsNoTracking()`.

Operaciones de Escritura Eficientes

Desactivación Masiva

Si la lógica de negocio requiere dar de baja productos antiguos, el servicio debe usar las funciones modernas de EF Core sin traer registros a RAM.

El servicio abstrae la complejidad. La vista web no necesita saber si EF Core usó `ExecuteUpdateAsync` o SQL crudo.

```
public async Task<int> DeactivateOldProductsAsync(Cancellation
{
    var limitDate = DateTime.UtcNow.AddYears(-2);

    // Actualización directa en BD, 1 sola query SQL
    var rowsAffected = await db.Products
        .Where(p => p.CreatedAt < limitDate && p.IsActive)
        .ExecuteUpdateAsync(setters => setters
            .SetProperty(p => p.IsActive, false),
            ct);

    return rowsAffected;
}
```


Sección 5:

Integración en Razor Pages

Consumiendo la arquitectura desde la Interfaz de Usuario

Consumo mediante Constructores Primarios

```
//  Constructor Primario en PageModel inyectando la Interfaz
public class IndexModel(IProductService productService) : PageModel
{
    // Solo datos necesarios para la UI
    public IReadOnlyList<ProductDto> Productos { get; set; } = null;

    public async Task OnGetAsync(CancellationToken ct)
    {
        // Delegamos al servicio. El PageModel no sabe
        // qué es EF Core ni un DbContext.
        Productos = await productService.GetActiveProductsAsync(ct);
    }
}
```

La Belleza del Desacoplamiento

-  **Menos código:** Gracias a C# 14, no hay campos privados ni constructores extensos.
-  **Inyección Limpia:** La página web solicita el `IProductService`. El contenedor sabe que debe instanciar `ProductService` y le inyecta su `AppDbContext` automáticamente. Es una cascada mágica.

Manejando Mutaciones (Patrón PRG)

Post-Redirect-Get

Las páginas Razor solo deben orquestar la petición HTTP, verificar validaciones (`ModelState`), delegar el comando al servicio y redirigir.

Nunca devuelvas `Page()` en un éxito; usa `RedirectToPage()` para evitar envíos dobles de formulario (PRG).

```
public async Task<IActionResult> OnPostAsync(CancellationToken
{
    if (!ModelState.IsValid) return Page();

    // Servicio ejecuta toda la lógica de alta de DB
    bool success = await productService.CreateProductAsync(
        Input.Name, Input.Price, ct);

    if (!success)
    {
        ModelState.AddModelError("", "Error al crear");
        return Page();
    }

    // Éxito: Redirigir
    return RedirectToPage("../Index");
}
```


Vistas Html Puras y Tipadas

```
@page  
@model IndexModel  
  
class="grid">  
    @foreach (var item in Model.Productos)  
    {  
        class="card">  
            @item.Name  
            @item.Price.ToString("C")  
  
        }  
    }
```

Una capa tonta por diseño

Al tener la lógica aislada en Servicios e Inyectada en el PageModel, el archivo .cshtml queda como un lienzo puro para UI. Cero reglas de negocio, cero consultas LINQ anidadas, 100% mantenibilidad.

Sección 6:

¿Por qué tanto esfuerzo?

Los verdaderos beneficios de la Inyección de Dependencias

El Verdadero Valor de **la Arquitectura**



Testeabilidad (Unit Tests)

Al depender de una interfaz (`IProductService`), puedes inyectar un "Mock" o simulador en tus Pruebas Unitarias. Puedes probar si la página Razor responde bien sin necesidad de montar una base de datos de SQLite real.



Reusabilidad Extrema

Si mañana quieres añadir una App Móvil, puedes crear un Controlador Web API e inyectarle el MISMO `ProductService`. No tienes que duplicar ninguna regla de negocio.

¿Preguntas?

El pegamento new desaparece; el framework orquesta.



Arquitectura .NET 10 | Servicios y DI